

Entrenamiento de Modelos Generativos

Hinton, máxima verosimilitud, KL divergence y MNIST

Maestría en Inteligencia Artificial

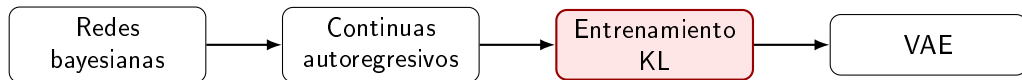
Universidad ORT Uruguay

15 de setiembre de 2026

Objetivos de la clase

- Entender qué significa entrenar un modelo generativo.
- Distinguir entrenamiento (con x completo) de generación secuencial.
- Relacionar máxima verosimilitud con KL divergence.
- Ver por qué un Bernoulli se entrena con binary cross-entropy, no con MSE.
- Preparar el práctico de perceptrón/MLP generativo sobre MNIST.

Dónde estamos



Ya tenemos formas de escribir $p_{\theta}(x)$. Hoy vemos cómo elegir θ .

Lo que ya tenemos

En Clase 3 el modelo autorregresivo predice la conjunta por regla de cadena:

$$p(x_1, \dots, x_n) = p(x_1) p(x_2 \mid x_1) \cdots p(x_n \mid x_{<n})$$

En Hinton, MNIST son 784 variables Bernoulli. El orden suele ser de izquierda a derecha y de arriba abajo, pero podría ser otro.

Las tablas condicionales crecen demasiado. Por eso pasamos a una función:

$$X_i \mid x_{<i} \sim \mathcal{B}(f_i(x_{<i}; w_i)), \quad f_i : \{0, 1\}^{i-1} \rightarrow [0, 1]$$

Los datos vienen de una distribución desconocida:

$$x \sim p_{\text{data}}(x)$$

Elegimos una familia de modelos:

$$p_{\theta}(x)$$

Entrenar significa elegir:

$$\theta^*$$

para que p_{θ} se parezca a p_{data} .

Likelihood

Dado un dataset:

$$T = \{x^{(1)}, \dots, x^{(m)}\}$$

si asumimos observaciones independientes:

$$p_{\theta}(T) = \prod_{j=1}^m p_{\theta}(x^{(j)})$$

Máxima verosimilitud:

$$\theta^* = \arg \max_{\theta} \prod_{j=1}^m p_{\theta}(x^{(j)})$$

Trabajar con productos es incómodo. Tomamos logaritmo:

$$\log p_{\theta}(T) = \sum_{j=1}^m \log p_{\theta}(x^{(j)})$$

Como log es monótona:

$$\arg \max_{\theta} p_{\theta}(T) = \arg \max_{\theta} \log p_{\theta}(T)$$

Maximizar likelihood equivale a maximizar log-likelihood.

Negative log-likelihood

En optimización solemos minimizar:

$$\mathcal{L}(\theta; T) = -\frac{1}{m} \sum_{j=1}^m \log p_{\theta}(x^{(j)})$$

Esta pérdida penaliza asignar baja probabilidad a datos observados.

Lectura generativa

Un buen modelo generativo debe hacer probable lo que aparece en el dataset.

KL divergence

Un modelo generativo aprende una distribución. Por eso comparar p y q con KL tiene sentido.

Para distribuciones discretas:

$$\text{KL}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

Equivalente:

$$\text{KL}(p\|q) = \mathbb{E}_{x \sim p} [\log p(x) - \log q(x)]$$

Mide cuánto se pierde al usar q para representar datos generados por p .

- No negatividad:

$$\text{KL}(p\|q) \geq 0$$

- Cero solo si las distribuciones coinciden:

$$\text{KL}(p\|q) = 0 \iff p = q$$

- No es simétrica:

$$\text{KL}(p\|q) \neq \text{KL}(q\|p)$$

- No es una distancia métrica.

Por qué no es simétrica

$$\text{KL}(p\|q) = \mathbb{E}_{x \sim p} [\log p(x) - \log q(x)]$$

La expectativa se toma sobre muestras de p .

- Si $p(x)$ es grande y $q(x)$ es muy chico, la penalización es alta.
- Si $p(x) = 0$, ese punto no aporta al promedio aunque $q(x) > 0$.

El orden importa porque decide desde qué distribución miramos el error.

Entrenar como minimizar KL

Queremos:

$$p_{\theta}(x) \approx p_{\text{data}}(x)$$

Una forma natural:

$$\theta^{\star} = \arg \min_{\theta} \text{KL}(p_{\text{data}} \| p_{\theta})$$

Desarrollamos:

$$\text{KL}(p_{\text{data}} \| p_{\theta}) = \mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\text{data}}(x) - \log p_{\theta}(x)]$$

De KL a máxima verosimilitud

$$\text{KL}(p_{\text{data}} \| p_{\theta}) = \mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\text{data}}(x)] - \mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\theta}(x)]$$

El primer término no depende de θ .

Entonces:

$$\arg \min_{\theta} \text{KL}(p_{\text{data}} \| p_{\theta}) = \arg \max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\theta}(x)]$$

Resultado

Minimizar $\text{KL}(p_{\text{data}} \| p_{\theta})$ equivale a maximizar log-likelihood.

No conocemos p_{data} y no podemos calcular la esperanza real. La estimamos por Monte Carlo con muestras:

$$T = \{x^{(1)}, \dots, x^{(m)}\}$$

$$\theta^* \approx \arg \max_{\theta} \frac{1}{m} \sum_{j=1}^m \log p_{\theta}(x^{(j)})$$

Eso es minimización de riesgo empírico para $\ell(\theta, x) = -\log p_{\theta}(x)$.

En la práctica: minibatch y SGD

La pérdida empírica:

$$\mathcal{L}(\theta; T) = -\frac{1}{|T|} \sum_{x \in T} \log p_{\theta}(x)$$

vale para **cualquier** modelo generativo, no solo para los autorregresivos.

En entrenamiento:

- no usamos T entero en cada paso;
- muestreamos un minibatch;
- actualizamos θ con SGD.

Esta pérdida, en un modelo AR

La NLL sirve para cualquier p_θ . En un modelo autorregresivo:

$$p_\theta(x) = \prod_{i=1}^n p_\theta(x_i \mid x_{<i})$$

$$\log p_\theta(x) = \sum_{i=1}^n \log p_\theta(x_i \mid x_{<i})$$

Así es como expresamos $\log p_\theta(x)$ dentro de $\arg \min_\theta -\mathbb{E}[\log p_\theta(x)]$.

Para un ejemplo:

$$\ell(\theta, x) = - \sum_{i=1}^n \log p_{\theta}(x_i \mid x_{<i})$$

Para un dataset:

$$\mathcal{L}(\theta; T) = - \frac{1}{|T|} \sum_{x \in T} \sum_{i=1}^n \log p_{\theta}(x_i \mid x_{<i})$$

Si $x_i \in \{0, 1\}$:

$$X_i \mid x_{<i} \sim \mathcal{B}(p_i)$$

$$p_{\theta}(x_i \mid x_{<i}) = p_i^{x_i} (1 - p_i)^{1-x_i}$$

Por lo tanto:

$$-\log p_{\theta}(x_i \mid x_{<i}) = -x_i \log p_i - (1 - x_i) \log(1 - p_i)$$

Binary cross-entropy

La pérdida Bernoulli es:

$$\text{BCE}(x_i, p_i) = -x_i \log p_i - (1 - x_i) \log(1 - p_i)$$

Para todo el vector:

$$\ell(\theta, x) = \sum_{i=1}^n \text{BCE}(x_i, p_i)$$

BCE no aparece por magia: es negative log-likelihood Bernoulli.

Por qué no MSE

La pérdida tiene que estar alineada con lo que el modelo produce: una probabilidad.

- MSE compara dato con dato.
- BCE compara un 0/1 con una probabilidad, usando logaritmos.

Si el pixel real es 1, queremos p_i cerca de 1. Si es 0, queremos p_i cerca de 0.

Además tiene que ser derivable: SGD necesita un gradiente.

Parametrizamos:

$$p_i = \text{sigmoid}(w_i^\top x_{<i} + b_i)$$

$$\text{sigmoid}(a) = \frac{1}{1 + \exp(-a)}$$

Entonces el modelo completo es:

$$p_\theta(x) = \prod_{i=1}^n \mathcal{B}(x_i; \text{sigmoid}(w_i^\top x_{<i} + b_i))$$

Entrenar no es generar

Dos etapas distintas:

Entrenamiento

- tenemos la imagen completa x ;
- calculamos todas las p_i de una vez;
- comparamos $\hat{x} = (p_1, \dots, p_n)$ contra el x real.

Generación

- no hay futuro disponible;
- muestreo $x_1 \sim \mathcal{B}(p_1)$;
- recién ahí puedo calcular p_2 ;
- igual que un chat: token a token.

Desafío del modelo AR

Puedo parametrizar la conjunta para entrenar, pero no para predecir. Necesito la posición $i - 1$ para dar la i . Ni un Transformer evita eso.

Forward pass de entrenamiento

Como x está completo, un solo producto alcanza:

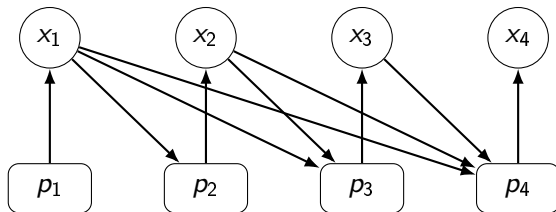
$$z = Wx + b, \quad \hat{x} = \text{sigmoid}(z)$$

W es estrictamente triangular: SGD no debe usar el futuro.

$$W = \begin{pmatrix} 0 & 0 & 0 & 0 \\ w_{21} & 0 & 0 & 0 \\ w_{31} & w_{32} & 0 & 0 \\ w_{41} & w_{42} & w_{43} & 0 \end{pmatrix}$$

$\hat{x}_i$ estima $p(x_i \mid x_{<i})$. Comparamos $\hat{x}$ contra el x real.

Hinton: belief network autorregresiva



$$p(x_1, x_2, x_3, x_4) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2)p(x_4 \mid x_1, x_2, x_3)$$

Costo en parámetros

Para cada posición i :

$$w_i \in \mathbb{R}^{i-1}$$

Cantidad total de pesos:

$$\sum_{i=1}^n (i-1) = \frac{n(n-1)}{2}$$

$$O(n^2)$$

Para imágenes, n puede ser grande: MNIST tiene $28 \times 28 = 784$ píxeles.

Podemos reemplazar el perceptrón por una red:

$$h_i = \text{sigmoid}(V_i x_{<i} + v_i)$$

$$p_i = \text{sigmoid}(a_i^\top h_i + b_i)$$

- Más expresividad: se puede jugar con la dimensión oculta.
- Captura interacciones no lineales.
- Aumenta costo si usamos una red distinta por posición.

Esta idea aparece en Bengio y Bengio (2000): pasar el perceptrón a un modelo multicapa.

Compartir parámetros: idea NADE

NADE reduce parámetros compartiendo una representación oculta:

$$h_i = \text{sigmoid}(V_{:, < i} x_{< i} + v)$$

$$p_i = \text{sigmoid}(a_i^\top h_i + b_i)$$

Idea

Las posiciones comparten parte de la computación, pero mantienen salidas específicas para cada x_i .

MNIST como vector binario

MNIST contiene imágenes de dígitos:

$$28 \times 28$$

Podemos aplanar cada imagen:

$$x \in [0, 1]^{784}$$

Para un modelo Bernoulli simple:

$$x_i \in \{0, 1\}$$

En la práctica podemos binarizar o interpretar intensidades como probabilidades.

Orden de los píxeles

Un modelo autorregresivo necesita un orden:

$$x_1, x_2, \dots, x_{784}$$

Ejemplos:

- barrido fila por fila;
- barrido columna por columna;
- orden aleatorio fijo;
- orden aprendido o estructurado.

En imágenes el orden es arbitrario. En lenguaje, no: el orden es parte del dato.

Para cada imagen x :

$$\ell(\theta, x) = \sum_{i=1}^{784} \text{BCE}(x_i, \text{sigmoid}(f_i(x_{<i}; \theta)))$$

Minimizamos el promedio en el dataset:

$$\mathcal{L}(\theta; T) = \frac{1}{|T|} \sum_{x \in T} \ell(\theta, x)$$

Generar una imagen

Una vez entrenado:

1. Calcular p_1 y muestrear x_1 .
2. Calcular $p_2 = p(x_2 = 1 \mid x_1)$ y muestrear x_2 .
3. Continuar hasta x_{784} .

$$x_i \sim \mathcal{B}(p_i)$$

Generar es secuencial. Entrenar usa el x real completo.

Evaluar durante el práctico

Medidas útiles:

- pérdida de entrenamiento;
- pérdida de validación;
- muestras generadas;
- comparación contra un baseline independiente:

$$p(x) = \prod_i p(x_i)$$

Cuidado

Buenas muestras visuales no siempre implican buena likelihood, y buena likelihood no siempre implica buenas muestras.

Limitaciones del enfoque autorregresivo

- Generación secuencial: cada píxel depende de los anteriores.
- El orden puede ser arbitrario.
- Capturar estructura global puede requerir muchas dependencias.
- Muestrear puede ser lento.

Otra familia de modelos introduce variables latentes continuas:

$$z \rightarrow x$$

Un modelo generativo latente:

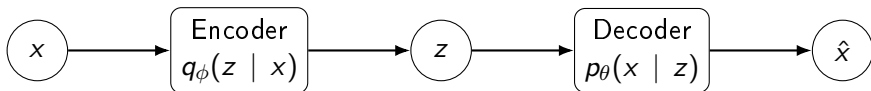
$$z \sim \mathcal{N}(0, I)$$
$$x \sim p_{\theta}(x \mid z)$$

La marginal:

$$p_{\theta}(x) = \int p_{\theta}(x \mid z) p(z) dz$$

Esta integral suele ser intratable en modelos neuronales.

Autoencoder variacional: idea



La próxima clase: cómo entrenar este modelo usando una cota variacional.

Dos problemas que vienen

Para entrenar VAE necesitamos resolver:

- qué loss usar si $p_{\theta}(x)$ tiene una integral intratable;
- cómo hacer backpropagation a través de una muestra:

$$z \sim q_{\phi}(z \mid x)$$

La respuesta incluirá:

- ELBO;
- KL entre gaussianas;
- reparameterization trick.

- Entrenar por máxima verosimilitud equivale a minimizar $KL(p_{\text{data}} \| p_{\theta})$.
- Esa esperanza se estima con datos, minibatches y SGD.
- Entrenar usa el x completo; generar es secuencial.
- Para Bernoulli, la NLL es BCE, no MSE.
- MNIST permite practicar estas ideas en un modelo generativo simple.

Idea central

La función de pérdida de un modelo generativo no se elige solo por conveniencia: sale de la distribución que el modelo asume para los datos.

$$\ell(\theta, x) = -\log p_{\theta}(x)$$

Cambiar p_{θ} cambia la pérdida.

1. Derivar la BCE desde una Bernoulli.
2. Mostrar que minimizar $KL(p_{\text{data}}||p_{\theta})$ equivale a maximizar log-likelihood.
3. Escribir la pérdida de un modelo autorregresivo de 4 variables binarias.
4. Calcular cuántos pesos tiene el modelo lineal para $n = 784$.
5. Explicar por qué un VAE necesita una aproximación variacional.

- B. J. Frey, G. E. Hinton y P. Dayan. *Does the Wake-Sleep Algorithm Learn Good Density Estimators?* NeurIPS, 1996.
- Y. Bengio y S. Bengio. *Modeling High-Dimensional Discrete Data with Multi-Layer Neural Networks*. NeurIPS, 2000.
- B. Uria et al. *Neural Autoregressive Distribution Estimation*. JMLR, 2016.
- Carl Doersch. *Tutorial on Variational Autoencoders*. arXiv, 2016/2021.
- Diederik Kingma y Max Welling. *Auto-Encoding Variational Bayes*. ICLR, 2014.